Module 1 - Python Fundamentals

1. Which built-in numeric type in Python automatically grows to hold arbitrarily large whole numbers?

- a) float
- b) int
- c) decimal.Decimal
- d) complex

A resposta correta é:

✅ **b) int**

Explicação

Em **Python 3**, o tipo **int** possui **precisão arbitrária (arbitrary precision)**. Isso significa que ele **cresce automaticamente para armazenar inteiros de qualquer tamanho**, limitado apenas pela memória disponível no sistema.

Ou seja, diferente de muitas linguagens (C, Java etc.), o int do Python **não tem limite fixo de bits (como 32 ou 64 bits)**.

Exemplo:

```
x = 10
y = 10**1000  # número extremamente grande

print(type(y))
print(y)
```

Python consegue armazenar esse número sem overflow.

Por que as outras estão erradas?

- **a) float** Usa padrão **IEEE-754 64 bits**, possui **limite de precisão**.
- **c) decimal.Decimal** É uma biblioteca para **aritmética decimal precisa**, mas **não é o tipo numérico built-in padrão**.
- **d) complex** Representa números complexos (a + bj), não inteiros arbitrários.

✔ Portanto, a resposta correta é:

b) int

---

2. Given text = "CYBER", what will text[::-1] return?

- ◯ a) "CYB"
- ◯ b) "REB"
- ◯ c) "REBYC"
- ◯ d) ["C", "Y", "B", "E", "R"]

A resposta correta é:

✅ **c) “REBYC”**

Explicação

A expressão:

```
text[::-1]
```

usa **string slicing** no formato:

```
[start : stop : step]
```

No caso:

```
[::-1]
```

significa:

- **start** → início da string (implícito)
- **stop** → final da string (implícito)
- **step = -1** → percorrer a string **de trás para frente**

Exemplo

```
text = "CYBER"
print(text[::-1])
```

Resultado:

```
REBYC
```

Passo a passo

Índice	Letra
0	C
1	Y
2	B
3	E
4	R

Invertendo:

```
R E B Y C
```

Resultado final:

```
"REBYC"
```

✔ **Alternativa correta: c) “REBYC”**

💡 **Dica importante (muito usada em Python e CTFs):**

[::-1] é o jeito **mais rápido de inverter uma string ou lista**.

```
reversed_text = text[::-1]
```

Isso aparece muito em:

- scripts de **malware analysis**

- **obfuscation de strings**
- desafios de **CTF / reversing**

3. Which collection offers average O(1) membership testing and guarantees uniqueness of its elements?

- a) list
- b) tuple
- c) set
- d) deque

A resposta correta é:

✅ **c) set**

Explicação

Em Python, **set** é uma coleção baseada em **hash table**, o que oferece:

- **Teste de pertencimento (membership test)** em **O(1)** em média

```
x in my_set
```

-
- **Elementos únicos** (não permite duplicatas)

Exemplo

```
s = {1, 2, 3, 3, 4}
```

```
print(s)
```

Resultado:

```
{1, 2, 3, 4}
```

O valor **3 duplicado é removido automaticamente**.

Comparação com as outras opções

Estrutura	Permite duplicados	Membership
list	✅ sim	O(n)
tuple	✅ sim	O(n)
set	❌ não	**O(1)**
deque	✅ sim	O(n)

✔ Portanto, a estrutura que **garante unicidade e possui membership O(1)** é:

c) set

💡 **Dica importante (muito usada em scripts de segurança / CTI):**

Sets são muito usados para:

- **deduplicar IOCs**
- **comparar listas de IPs/domínios**
- **detectar interseção entre feeds**

Exemplo real em threat intelligence:

```
ioc_feed1 = {"1.1.1.1", "2.2.2.2"}
ioc_feed2 = {"2.2.2.2", "3.3.3.3"}

common = ioc_feed1 & ioc_feed2
print(common)
```

Resultado:

```
{'2.2.2.2'}
```

4. In a for loop, the optional else block executes when...

- ◯ a) the loop completes without hitting break
- ◯ b) the loop encounters a break statement
- ◯ c) the loop raises an exception
- ◯ d) the loop iterates over an empty sequence

A resposta correta é:

✅ **a) the loop completes without hitting break**

Explicação

Em Python, um **for loop pode ter um bloco else**, que é executado **somente se o loop terminar normalmente**, ou seja:

- percorreu todos os elementos
- **não encontrou nenhum break**

Exemplo

```
nums = [1, 2, 3, 4]

for n in nums:
    if n == 5:
        print("Found!")
        break
else:
    print("Not found")
```

Saída:

```
Not found
```

O else executa porque **o loop terminou sem break**.

Caso com

break

```
nums = [1, 2, 3, 4]

for n in nums:
    if n == 3:
        print("Found!")
        break
else:
    print("Not found")
```

Saída:

```
Found!
```

O else **não executa**, pois o loop foi interrompido pelo break.

Resumo

Situação	else executa?
Loop termina normalmente	✅ Sim
break ocorre	❌ Não
Exceção ocorre	❌ Não

✔ Resposta correta: a) the loop completes without hitting break

---

```python
def add_item(item, bucket=[]):
    bucket.append(item)
    return bucket
```

5. Why is the following function dangerous?

- ◯ a) Default arguments are evaluated at every call, wasting memory.
- ◯ b) bucket is recreated on each call, losing previous items.
- ◯ c) The syntax is illegal; default values must be immutable.
- ◯ d) The same list is shared across calls, causing unexpected accumulation.

A resposta correta é:

✅ **d) The same list is shared across calls, causing unexpected accumulation.**

Explicação

Em Python, **argumentos padrão são avaliados apenas uma vez**, no momento em que a função é **definida**, e **não a cada chamada**.

No código:

```python
def add_item(item, bucket=[]):
    bucket.append(item)
    return bucket
```

A lista bucket é criada **uma única vez** e **reutilizada em todas as chamadas da função**.

Exemplo do problema

```
print(add_item(1))
print(add_item(2))
print(add_item(3))
```

Resultado:

```
[1]
[1, 2]
[1, 2, 3]
```

Mesmo sem passar bucket, a lista **continua acumulando itens**.

Isso acontece porque **o mesmo objeto lista é compartilhado entre chamadas**.

Forma correta de implementar

O padrão seguro é usar None:

```
def add_item(item, bucket=None):
    if bucket is None:
        bucket = []
    bucket.append(item)
    return bucket
```

Agora cada chamada cria uma nova lista.

Por que as outras opções estão erradas?

- **a)** errado → argumentos padrão **não são avaliados a cada chamada**
- **b)** errado → a lista **não é recriada**, ela é reutilizada
- **c)** errado → Python **permite valores mutáveis como default**, mas é perigoso

✔ **Resposta correta: d)**

---

6. What value does __name__ hold when a module is imported (not run directly)?

- a) "__main__"
- b) "__package__"
- c) the module's filename without .py
- d) None

A resposta correta é:

✅ **c) the module's filename without .py**

Explicação

Em Python, a variável especial **name** indica **como o módulo está sendo executado**.

Existem dois casos:

1️⃣ Quando o arquivo é executado diretamente

```
python script.py
```

Dentro do arquivo:

```
print(__name__)
```

Resultado:

```
__main__
```

2️⃣ Quando o arquivo é importado como módulo

Exemplo:

arquivo: tools.py

```
print(__name__)
```

arquivo: main.py

```
import tools
```

Saída:

```
tools
```

Ou seja, **name recebe o nome do módulo (nome do arquivo sem .py)**.

Uso clássico

Esse comportamento é usado no padrão:

```
if __name__ == "__main__":
    main()
```

Isso garante que o código **só executa quando o script é rodado diretamente**, e não quando é importado.

Por que as outras estão erradas?

- **a)** "**main**" → apenas quando o arquivo é executado diretamente
- **b)** "**package**" → é outra variável especial
- **d)** None → não é o comportamento de **name**

✔ Resposta correta: c)

---

7. Opening a file with mode "rb" instead of "r" is essential when you...

- ◯ a) need automatic newline translation
- ◯ b) must preserve raw bytes, such as an image or executable
- ◯ c) are reading UTF-8 text
- ◯ d) want Python to guess the encoding

A resposta correta é:

✅ **b) must preserve raw bytes, such as an image or executable**

Explicação

Em Python:

- **"r"** → **modo texto**
- **"rb"** → **modo binário**

Quando você usa **rb**, o arquivo é lido **como bytes brutos**, sem:

- conversão de encoding
- tradução de newline
- interpretação de caracteres

Isso é essencial para **arquivos binários**, como:

- imagens (.png, .jpg)
- executáveis (.exe, .bin)
- arquivos compactados (.zip)
- malware samples

- pacotes de rede
- hashes de arquivos

Exemplo

```
with open("malware.exe", "rb") as f:
    data = f.read()
```

Aqui data será um objeto **bytes**, preservando exatamente os dados do arquivo.

Diferença entre

"r"

e

"rb"

Modo	Tipo retornado	Uso
"r"	str	arquivos de texto
"rb"	bytes	arquivos binários

Exemplo prático (hashing)

Em **DFIR / malware analysis**, sempre usamos "rb":

```
import hashlib

with open("sample.exe", "rb") as f:
    sha256 = hashlib.sha256(f.read()).hexdigest()

print(sha256)
```

Se usar "r", o hash pode **quebrar ou mudar**.

✔ **Resposta correta: b)**

8. Which except clause correctly catches only a missing file and leaves other I/O errors untouched?

- ◯ a) except IOError:
- ◯ b) except FileNotFoundError:
- ◯ c) except OSError:
- ◯ d) except Exception:

A resposta correta é:

✅ **b) except FileNotFoundError:**

Explicação

FileNotFoundError é uma exceção **específica** que ocorre quando Python tenta abrir um arquivo que **não existe**.

Exemplo:

```
try:
    with open("data.txt", "r") as f:
        data = f.read()
except FileNotFoundError:
    print("File not found")
```

Aqui apenas o erro **arquivo inexistente** será capturado.

Por que as outras opções estão erradas?

a) except IOError:

Captura **muitos erros de I/O**, como:

- permissões
- disco cheio
- falhas de leitura

---

c) except OSError:

Ainda **mais genérico**, cobre vários erros do sistema operacional.

---

d) except Exception:

Captura **quase qualquer erro**, o que não é recomendado quando você quer tratar um caso específico.

Hierarquia simplificada

```
Exception
 └── OSError
      └── IOError (alias em Python 3)
           └── FileNotFoundError
```

Logo:

✔ **FileNotFoundError é o mais específico**.

Boa prática

Capturar **exceções específicas** torna o código:

- mais seguro
- mais previsível
- mais fácil de debugar

✔ **Resposta correta: b)**

9. Select the most Pythonic way to test whether a list items is empty.

- a) if len(items) == 0:
- b) if items == []:
- c) if not items:
- d) if items is None:

A resposta correta é:

✅ **c) if not items:**

Explicação

Em Python, coleções vazias são avaliadas como **False** em contexto booleano. Isso inclui:

- [] (lista vazia)
- {} (dicionário vazio)
- set()
- "" (string vazia)
- None

Portanto, a forma **mais Pythonic** é:

```
if not items:
    print("List is empty")
```

Exemplo

```
items = []

if not items:
    print("Empty list")
```

Saída:

```
Empty list
```

Por que as outras opções não são ideais?

a) if len(items) == 0:

Funciona, mas é **menos idiomático em Python**.

b) if items == []:

Comparação direta com lista vazia é **desnecessária**.

d) if items is None:

Verifica se a variável é **None**, não se a lista está vazia.

Regra Pythonic importante

Em Python, preferimos **truthiness checks**:

```
if items:        # lista não vazia
if not items:    # lista vazia
```

✔ **Resposta correta: c)**

10. A @classmethod differs from a regular instance method because it...

- ◯ a) receives the class (cls) as its first argument rather than the instance (self)
- ◯ b) cannot access any attributes
- ◯ c) is automatically static and immutable
- ◯ d) is only available when inheriting from object

A resposta correta é:

✅ **a) receives the class (cls) as its first argument rather than the instance (self)**

Explicação

Em Python existem **três tipos principais de métodos em classes**:

Tipo	Primeiro argumento	Acesso
Instance method	self	instância
Class method	cls	classe
Static method	nenhum automático	nenhum específico

Método de instância (normal)

```
class MyClass:
    def method(self):
        print(self)
```

self representa **a instância do objeto**.

Class method

```
class MyClass:

    @classmethod
    def method(cls):
        print(cls)
```

Aqui o primeiro argumento é **cls**, que representa **a própria classe**.

Pode ser chamado assim:

```
MyClass.method()
```

ou

```
obj = MyClass()
obj.method()
```

Exemplo prático

```
class User:

    total_users = 0

    def __init__(self):
        User.total_users += 1

    @classmethod
    def count(cls):
        return cls.total_users
```

Uso:

```
User()
User()
print(User.count())
```

Resultado:

```
2
```

Por que as outras opções estão erradas?

b) errado → classmethods **podem acessar atributos da classe**

c) errado → isso descreve mais um **@staticmethod**

d) errado → não depende de herdar de object

✔ **Resposta correta: a)**